

COURSE

Основы программирования JS

Библиотека Node.js и пакетный менеджер NPM

shesnokov@gmail.com

ОГЛАВЛЕНИЕ

План лекции ***3***

Теоретическая часть ***3***

Библиотека Node.js. **3**

Установка Node.js 4

Начало работы (режим REPL) 5

Запуск JavaScript кода 6

Файлы ввода/вывода с Node.js 6

Создание веб-сервера с помощью Node.js 7

Модульность приложений 8

Пакетный менеджер NPM. **10**

Добавление зависимостей 10

Добавление зависимостей для разработки 11

NPM-скрипты 11

Ограничение области видимости и приватные пакеты 11

Литература и другие источники ***13***

Вопросы для самопроверки ***13***

ПЛАН ЛЕКЦИИ

1. Обзор сервисов и IDE для разработки на Jscript.
2. Библиотека Node.js.
3. Пакетный менеджер NPM.
4. Редактор MS Visual Code.
5. IDE JetBrains WebStorm.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

БИБЛИОТЕКА NODE.JS.

Node.js — это опенсорсная кроссплатформенная среда выполнения для JavaScript, которая работает на серверах. С момента выпуска этой платформы в 2009 году она стала чрезвычайно популярной и в наши дни играет весьма важную роль в области веб-разработки.

Платформа Node.js построена на базе JavaScript движка V8 от Google, который используется в браузере Google Chrome. Данная платформа, в основном, используется для создания веб-серверов, однако сфера её применения этим не ограничивается.

Node.js предлагает вам возможность писать невероятно производительный серверный код с использованием JavaScript.

Преимущества Node.js:

- **Скорость.** Одной из основных привлекательных особенностей Node.js является скорость. JavaScript-код, выполняемый в среде Node.js, может быть в два раза быстрее, чем код, написанный на компилируемых языках, вроде C или Java, и на порядки быстрее интерпретируемых языков наподобие Python или Ruby. Причиной подобного является неблокирующая архитектура платформы, а конкретные результаты зависят от используемых тестов производительности, но, в целом, Node.js — это очень быстрая платформа.
- **Простота.** Платформа Node.js проста в освоении и использовании. Особенно это заметно в сравнении с некоторыми другими серверными платформами.
- **Использование JavaScript.** В среде Node.js выполняется код, написанный на JavaScript. Это означает, что миллионы фронтенд-разработчиков, которые уже пользуются JavaScript в браузере, могут писать и серверный, и клиентский код на одном и том же языке программирования без необходимости изучать совершенно новый инструмент для перехода к серверной разработке. В браузере и на сервере используются одинаковые концепции языка. Кроме того, в Node.js можно оперативно переходить на использование новых стандартов ECMAScript по мере их реализации на платформе. Для этого не нужно ждать до тех пор, пока пользователи обновят браузеры, так как Node.js — это серверная среда, которую полностью контролирует разработчик. В результате новые возможности языка оказываются доступными при установке поддерживающей их версии Node.js.
- **Движок V8.** В основе Node.js, помимо других решений, лежит опенсорсный JavaScript-движок V8 от Google, применяемый в браузере Google Chrome и в других браузерах. Это означает, что Node.js пользуется наработками тысяч инженеров, которые сделали среду выполнения JavaScript Chrome невероятно быстрой и продолжают работать в направлении совершенствования V8.

- **Асинхронность.** В традиционных языках программирования (C, Java, Python, PHP) все инструкции, по умолчанию, являются блокирующими, если только разработчик явным образом не позаботится об асинхронном выполнении кода. В результате если, например, в такой среде, произвести сетевой запрос для загрузки некоего JSON-кода, выполнение потока, из которого сделан запрос, будет приостановлено до тех пор, пока не завершится получение и обработка ответа. Асинхронные механизмы позволяют единственному Node.js-серверу одновременно обрабатывать тысячи подключений, не нагружая при этом программиста задачами по управлению потоками и по организации параллельного выполнения кода. Подобные вещи часто являются источниками ошибок.
- **Библиотеки.** Благодаря простоте и удобству работы с менеджером пакетов для Node.js, который называется npm, экосистема Node.js прямо-таки процветает. Сейчас в реестре npm имеется более полумиллиона опенсорсных пакетов, которые может свободно использовать любой Node.js-разработчик.

Рассмотрев некоторые основные особенности платформы Node.js, опробуем её в действии. Начнём с установки.

Установка Node.js

Последнюю версию Node.js вы можете найти на официальном сайте: https://nodejs.org/en/. На главной странице сайта предлагается скачать последнюю версию продукта и последнюю стабильную версию (так называемую «долгоиграющую» LTS сборку). Запускается инсталлятор или устанавливается пакет. Node.js является кроссплатформенным приложением, поэтому вы можете установить продукт под Windows, OSX или Linux. Интерфейс и функциональные возможности ничем отличаться не будут.

При таком подходе довольно легко начать работу, но, если позже вы захотите добавить в систему больше версий Node.js, лучше сразу начать использовать **nvm (node version manager)** - диспетчером версий Node.js. После его установки под выбранную платформу вы сможете использовать очень простой CLI API для смены версии Node.js.

Установка различных версий Node.js:

nvm install 8.5

Затем, если вы хотите проверить в работе экспериментальную версию:

nvm install 9

Чтобы убедиться, что у вас установлена и запущена Node.js, выполните (перед version ставиться два тире):

node --version

Если все в порядке, последняя команда вернет номер версии текущего активного бинарного файла Node.js.

Если вы работаете над проектом, поддерживающим Node.js v8, вы можете начать использовать эту версию с помощью следующей команды:

nvm use 8

Затем вы можете переключиться на Node.js v9 с помощью той же самой команды:

nvm use 9

С тех пор как был сформирован Node.js Foundation, Node.js имеет план релизов. Это очень похоже на другие проекты Linux Foundation. Это означает, что есть два релиза: стабильный и экспериментальный. В Node.js стабильными версиями с долговременной поддержкой (**LTS**) являются те, которые начинаются с четных чисел (4, 6, 8, …). Экспериментальные версии нумеруются нечетными числами (5, 7, 9, …).

Начало работы (режим REPL)

Чтобы начать работу с Node.js, давайте попробуем её в консоли! Режим работы с Node.js через консоль называется **REPL**. Здесь задействуется модуль repl, представляющий собой реализацию **Real-Eval-Print-Loop (Чтение-оценка-печать-цикл, REPL)**, которая доступна как в виде автономной программы, так и встроенной в другие приложения.

Нижеследующие команды поддерживаются всеми экземплярами REPL:

- **.break** - в процессе ввода многострочного выражения ввод команды .break (или нажатие комбинации клавиш <Ctrl>-C) прервет последующий ввод или обработку этого выражения.
- **.clear** - сбрасывает context REPL до пустого объекта и стирает все многострочные выражения, которые вводятся в данный момент.
- **.exit** – закрывает все стримы I/O, заставляя REPL завершиться.
- **.help** – показывает список специальных команд.
- **.save** – сохраняет текущую сессию REPL в файл > .save ./file/to/save.js.
- **.load** – загружает файл в текущую сессию REPL > .load ./file/to/load.js.
- **.editor** – входит в режим правки (<Ctrl>-D для завершения, <Ctrl-C> для отмены)

Следующие комбинации клавиш в REPL работают так:

- **<Ctrl>-C** – при единоразовом нажатии имеет такой же эффект, как команда .break. При двойном в пустой строке – такой же эффект, как и .exit.
- **<Ctrl>-D** – имеет такой же эффект, как команда .exit.
- **<tab>** - при нажатии в пустой строке отображает глобальные и локальные переменные. При нажатии во время ввода показывает варианты автозавершения.

Итак, чтобы начать работу нужно открыть консоль (терминал) и запустить Node.js, просто набрав node:

> node

>

Можно попробовать что-то напечатать:

> node

> console.log('hello from Node.js')

После нажатия Enter вы получите следующее:

> console.log('hello from Node.js')

hello from Node.js

undefined

Режим REPL очень хорош для проверки однострочных конструкций, но с Node.js удобнее работать, запуская на выполнения приложения. Под **приложением** в данном случае понимается программный код, оформленный в виде скрипта .js. Попробуем создать подобное приложение.

Запуск JavaScript кода

Начнем с создания файла hello.js. Откройте свою IDE (Atom, Sublime, VS Code или просто блокнот - выбор за вами), создайте новый файл и сохраните его с именем hello.js. Затем скопируйте в него следующий фрагмент кода:

```
// hello.js
console.log('hello from Node.js')
```

Чтобы запустить этот файл, вы должны снова открыть свой терминал и перейти в каталог, в котором размещён hello.js. Как только вы успешно переместитесь в нужное место, запустите файл, используя команду

```
> node hello.js
```

Вы увидите, что эта команда будет выдавать тот же результат, что и раньше, выводя строку непосредственно в терминале.

Файлы ввода/вывода с Node.js

Запуск чистого JavaScript — это здорово, но не очень полезно. Поэтому в Node.js есть огромное количество библиотек (модулей) для того, чтобы делать реальные вещи. В данном примере мы откроем файл с записями и будем его обрабатывать.

Создайте файл ***example_log.txt*** и внесите в него некоторое количество записей:

```
2020-08-09T13:50:33.166Z A 2
2020-08-09T13:51:33.166Z B 1
2020-08-09T13:52:33.166Z C 6
2020-08-09T13:53:33.166Z B 8
2020-08-09T13:54:33.166Z B 5
```

Что означает этот файл, не так важно, но нужно знать, что каждая запись содержит дату, букву и значение. Мы хотим соотнести каждую букву с соответствующим значением. Нам нужно считать содержание файла. Для это создадим файл my_parser.js и внесем в него программный код:

```
// Загружаем модуль файловой системы
var fs = require('fs');
// Считываем содержание файла в память
fs.readFile('example_log.txt', function (err, logData) {
    // Если возникла ошибка, мы кидаем исключение
    // и программа заканчивается
    if (err) throw err;
    // logData это объект типа Buffer, переводим в строку
        var text = logData.toString();
});
```

К счастью, Node.js сильно облегчает обработку содержания файла с помощью встроенного модуля ***filesystem (fs)***. Модуль fs имеет функцию readFile, которая берёт путь к файлу и обратный вызов. Обратный вызов будет исполнен, когда файл будет полностью прочитан. Данные файла попадают в форме типа Buffer, что является набором битов. Мы можем конвертировать в строку с помощью функции ***toString()***. Теперь добавим парсер (он написан на чистом JavaScript).

```
// загружаем модуль filesystem(fs)
var fs = require('fs');
```

```
// считываем содержание файла в память
fs.readFile('example_log.txt', function (err, logData) {
    // Если возникла ошибка, мы кидаем исключение
      // и программа заканчивается
   if (err) throw err;
    // logData имеет тип Buffer, конвертируем в строку
   var text = logData.toString();
 var results = {};
 // Разбиваем текст на массив из строчек
var lines = text.split('\n');
 lines.forEach(function(line) {
      var parts = line.split(' ');
      var letter = parts[1];
       var count = parseInt(parts[2]);
      if(!results[letter]) {
          results[letter] = 0;
      }
      results[letter] += parseInt(count);
});
 console.log(results);
// { A: 2, B: 14, C: 6 }
});
```

Когда файл будет аргументом команды node, результат распечатается, и будет осуществлён выход.

```
$ node my_parser.js
{ A: 2, B: 14, C: 6 }
```

Создание веб-сервера с помощью Node.js

Как было уже сказано, Node ничего не может делать сам по себе. Один из встроенных модулей позволяет легко создать простой HTTP сервер, пример его использования приведен на главной странице официального сайта.

Создадим файл ***mywebserver.js*** и внесем в него следующий код:

```
var http = require('http');
http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/plain'});
  res.end('Hello World\n');
}).listen(8080);
console.log('Server running on port 8080.');
```

Это базовый веб-сервер, а не сервер с полным функционалом. Он не может обслуживать картинки или HTML-файлы. На самом деле, какой бы вы ни отправили запрос, он возвратит “Hello World”. Однако, вы можете запустить этот скрипт, перейти по адресу http://localhost:8080 в своем браузере и увидеть этот текст.

> node mywebserver.js

Вы уже могли заметить, что ваше приложение на node.js больше не останавливается. Всё потому, что вы создали веб-сервер, и ваше приложение на node.js отвечает на запросы до тех пор, пока вы сами его не остановите.

Если вы хотите сделать веб-сервер с полной функциональностью, вам придётся проделать некоторый объем работы: нужно будет проверить все запросы, считать подходящие файлы и отправить вывод обратно. Есть и хорошие новости. Люди уже проделали эту тяжелую работу за вас.

Модульность приложений

Теперь у нас есть файл index.js, поэтому пришло время перейти на следующий уровень! Давайте создадим что-то более сложное, разделив наш исходный код на несколько JavaScript-файлов с целью удобочитаемости и поддерживаемости. Чтобы начать работу, вернитесь в свою IDE и создайте следующую структуру каталогов (с пустыми файлами), но пока не трогайте package.json, его можно сгенерировать автоматически на следующем шаге:

```
├── app
|   ├── calc.js
|   └── index.js
├── index.js
└── package.json
```

Каждый проект Node.js начинается с создания файла **package.json**. Вы можете думать о нем как о **JSON-представлении приложения и его зависимостей**. Он содержит имя вашего приложения, автора (вас) и все зависимости, необходимые для запуска приложения.

Вы можете интерактивно генерировать файл package.json с помощью команды **npm init** в терминале. После запуска команды требуется ввести некоторые данные, например, имя вашего приложения, версию, описание и так далее. Не нужно беспокоиться, просто нажимайте Enter, пока не получите сформированный JSON и вопрос **is it ok?**. Нажмите Enter в последний раз и вот: ваш package.json был автоматически сгенерирован и помещен в папку вашего приложения. Если вы откроете этот файл в своей IDE, он будет очень похож на фрагмент кода ниже.

```
// package.json
{
  "name": "node-hero",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node index.js"
  },
  "author": "", "license": "ISC"
}
```

Хорошей практикой является добавление стартового скрипта в ваш пакет package.json. Как только вы это сделаете, как показано в примере выше, вы можете запустить приложение с помощью команды **npm start**. Это очень удобно при переносе и развертывании приложений.

Теперь давайте вернемся к первому созданному вами файлу под названием index.js. Лучше оставить этот файл очень компактным: только подключение самого приложения (файл index.js из подкаталога /app, созданного ранее). Скопируйте следующий код в свой файл index.js и сохраните:

```
// index.js
require('./app/index')
```

Теперь пришло время приступить к созданию реального приложения. Откройте файл index.js из папки /app, чтобы создать очень простой пример: добавление массива чисел. В этом случае файл index.js будет содержать только числа, которые мы хотим добавить, а логика, требующая вычислений, должна быть помещена в отдельный модуль.

Вставьте этот код в файл index.js в каталоге /app.

```
// app/index.js
const calc = require('./calc')
const numbersToAdd = [
  3,
  4,
  10,
  2
]

const result = calc.sum(numbersToAdd)
console.log(`The result is: ${result}`)
```

Теперь вставьте фактическую бизнес-логику в файл calc.js, который можно найти в той же папке.

```
// app/calc.js
function sum (arr) {
  return arr.reduce(function(a, b) {
    return a + b
  }, 0)
}

module.exports.sum = sum
```

Чтобы проверить, всё ли вы сделали правильно, сохраните эти файлы, откройте терминал и введите **npm start** или **node index.js**. Если вы все сделали правильно, вы получите ответ: **19**. Если что-то пошло не так, внимательно просмотрите лог в консоли и найдите проблему на его основе.

На основе данного примеры была рассмотрена возможность использования модулей при разработке кода и организации модульной структуры приложения Node.js.

Пакетный менеджер NPM.

NPM - это менеджер пакетов, используемый Node.js-приложениями. В нём вы можете найти массу готовых модулей, поэтому вам не нужно «изобретать колесо». Модули создаются разработчиками по всему миру и несут различную функциональность. NPM похож на Maven для Java или Composer для PHP.

Существует два основных интерфейса, с которыми вы будете взаимодействовать: **сайт NPM** и **набор инструментов командной строки (CLI)**. И веб-сайт, и CLI используют один и тот же реестр, чтобы искать и отображать модули. Сайт NPM можно найти по адресу https://npmjs.com. Здесь вы можете зарегистрироваться как новый пользователь или поискать нужные пакеты.

Чтобы запустить CLI, достаточно написать:

> npm

Обратите внимание, что NPM поставляется вместе с бинарным файлом Node.js, поэтому вам не нужно его устанавливать, однако если вы хотите использовать определенную версию NPM, вы можете его обновить. Например, если вы хотите установить NPM версии 4, вы можете сделать это с помощью: npm install npm@4 -g.

Чтобы посмотреть версию установленного NPM, нужно запустить команду:

> npm --version

Добавление зависимостей

В этом разделе вы узнаете, как добавлять рантайм зависимости к вашему приложению. Когда у вас есть файл package.json, вы можете добавить зависимости к вашему приложению. Давайте добавим одну. Попробуйте следующее:

> npm install lodash --save

С помощью этой единственной команды мы достигли двух вещей: во-первых, **lodash** загружен и помещён в папку **node_modules**. Это папка, в которой будут находиться все ваши внешние зависимости. Обычно вы не хотите добавлять её в свою систему управления версиями, поэтому, если вы используете git, обязательно добавьте node_modules в файл .gitignore.

Давайте посмотрим, что происходит в файле package.json. Появилось новое свойство, называемое dependencies:

```
"dependencies": {
  "lodash": "4.6.1"
}
```

Это означает, что lodash версии 4.6.1 теперь установлен и готов к использованию. Обратите внимание, что NPM следует правилам SemVer (http://semver.org/) для версионирования пакетов.

Поскольку lodash готов к использованию, давайте посмотрим, как мы можем им воспользоваться. Вы можете сделать это так же, как и с вашим собственным модулем, но теперь вам не нужно указывать полный путь, достаточно только имени модуля:

```
// index.js
const _ = require('lodash')
_.assign({ 'a': 1 }, { 'b': 2 }, { 'c': 3 });
// → { 'a': 1, 'b': 2, 'c': 3 }
```

Добавление зависимостей для разработки

Когда вы собираете веб-приложения, вам может потребоваться минимизировать ваши JavaScript-файлы, объединить CSS-файлы и так далее. Модули, которые это сделают, будут выполняться только во время создания ресурсов, поэтому работающее приложение не нуждается в них.

Вы можете установить такие скрипты с помощью:

> npm install mocha --save-dev

После этого в вашем файле package.json появится новый раздел, называемый devDependencies. Все модули, которые вы устанавливаете с помощью --***save-dev***, будут описаны там, а также они будут помещены в тот же самый каталог ***node_modules***.

NPM-скрипты

NPM-скрипты — очень мощная концепция, с их помощью вы можете создавать небольшие утилиты или даже описывать сложные системы сборки. Наиболее распространенными являются скрипты **start** и **test**. С помощью start вы можете описать, как нужно запускать приложение, а test используется для запуска тестов. В вашем package.json они могут выглядеть примерно так:

```
"scripts": {
  "start": "node index.js",
  "test": "mocha test",
  "your-custom-script": "echo npm"
}
```

Что следует здесь отметить:

- **start**: просто описывает начальную точку для запуска вашего приложения, этот скрипт можно вызвать с помощью **npm start**.
- **test**: цель этого скрипта заключается в том, чтобы запускать ваши тесты: одно из удобств такого запуска тестов заключается в том, что в этом случае mocha не нужно устанавливать глобально, так как NPM будет искать её в папке node_modules/.bin, а mocha будет размещена там же. Запуск тестов может быть вызван с помощью **npm test**.
- **your-custom-script**: всё, что вы захотите (вы можете выбрать любое имя). Его можно вызвать с помощью **npm run your-custom-script** - не забывайте про run в этом случае.

Ограничение области видимости и приватные пакеты

Первоначально у NPM было глобальное пространство имен для названий модулей, и с более чем 250 000 модулями в реестре большинство простых имен уже заняты. Кроме того, глобальное пространство имен содержит только общедоступные модули.

В NPM урегулировали эту проблему внедрением пакетов с ограниченной областью видимости (scoped packages), они имеют следующий шаблон именования:

@myorg/mypackage

Вы можете устанавливать пакеты с ограниченной областью видимости так же, как и раньше:

> npm install @myorg/mypackage --save-dev

Они будут отображаться в вашем package.json следующим образом:

```
"dependencies": {
  "@myorg/mypackage": "^1.0.0"
```

}

Подключения пакетов с ограниченной областью видимости работает должным образом:

> require('@myorg/mypackage')

ЛИТЕРАТУРА И ДРУГИЕ ИСТОЧНИКИ

1. Пауэрс Ш. Изучаем Node. Переходим на сторону сервера. 2-е изд. Изд. Питер, 2017 – 304с.
2. Давид Хэррон. Node.js Разработка серверных веб-приложений на JavaScript. –М.: ДМК Пресс, 2012 – 144с.
3. Сухов.К.К. Node.js. Путеводитель по технологии. –М.: ДМК Пресс, 2015. - 416 с.
4. Кантелон М., Хартер, М. Головайчук TJ, Рейлих Н. Node.js в действии. – СПб.: Питер, 2015. – 448 с.
5. Аквино К., Ганди Т. Front-end. Клиентская разработка для профессионалов. Node.js, ES6, REST. Изд. Питер, 2017. – 512с.

ВОПРОСЫ ДЛЯ САМОПРОВЕРКИ

1. Что такое библиотека Node.js? Какие возможности она предоставляет?
2. Как переключиться в REPL режим и выполнить программный код JavaScript?
3. Основные команды REPL режима. Как выполнить многострочный код?
4. Как можно создать приложение для Node.js? Его основные составляющие.
5. Как подключить внутренние/внешние модули Node.js в ваш проект?
6. Что такое NPM? Основные команды для работы с npm.